Serial Python Minimalis: Tutorial #02-String

Ahmad R. T. Nugraha

ver. 21 April 2026

1 Definisi String

"String" (dengan tipe `str` dalam Python) adalah urutan karakter, biasanya mewakili data teks. Contoh string adalah pesan yang dapat dicetak di layar. String didefinisikan dengan menggunakan tanda kutip tunggal maupun ganda, dan dapat dialokasikan ke nama variabel seperti halnya angka. Jika kita tuliskan format string dengan tanda kutip begitu dan langsung dieksekusi, bagian output pada mode interaktif Python akan langsung memberikan hasil yang sama dengan yang ditulis pada input.

```
'sodium'
```

Output:

```
'sodium'
```

```
gas = "Carbon dioxide"
print(gas)
```

Output:

```
Carbon dioxide
```

String dapat digabungkan (*concatenated*) dengan operator + dan diulang dengan operator *:

```
'CH3' + 'CH2'*3 + 'CH3'
```

Output:

```
'CH3CH2CH2CH2CH3'
```

```
prefix = 'trans-'
prefix + 'but-2-ene'
```

Output:

```
'trans-but-2-ene'
```

String yang berada di antara tanda kutip disebut *string literal*. String semacam ini yang didefinisikan bersebelahan satu sama lain akan digabungkan secara otomatis (tanpa memerlukan +).

```
'Na' 'Cl'
```

Output:

```
'NaCl'
```

Perhatikan bahwa tidak ada spasi yang ditambahkan ke dalam string gabungan tersebut dan biasanya akan berguna ketika membuat string yang panjang, yang dapat dipecah menjadi beberapa bagian pada baris terpisah asalkan keseluruhan ekspresi tersebut diapit dalam tanda kurung.

```
quote = ('For me physics represented an indefinite cloud of future '
         'potentialities which enveloped my life to come in silent '
         'symmetries and vast cosmic laws')
```

Karakter `escape`

Pemisah baris (*line breaks*) tidak disertakan dalam string. Untuk merepresentasikan baris baru dan karakter khusus lainnya seperti tab, kita perlu menggunakan karakter "penyelamatan" (*escape*), yang diawali dengan garis miring terbalik (*backslash*, '\'). Tabel 1 menunjukkan urutan *escape* (*escape sequences*) yang umum digunakan.

Tabel 1: Urutan *escape* Python yang umum digunakan.

Urutan *escape*	Makna
`\'`	Tanda kutip tunggal (')
`\"`	Tanda kutip ganda (")
`\n`	*Linefeed* (LF) / Baris baru
`\r`	*Carriage return* (CR)
`\t`	Tab horizontal
`\\`	Karakter *backslash* itu sendiri
`\u, \U, \N{}`	Karakter Unicode berdasarkan *code point*-nya

Karakter-karakter `escape` ini tidak akan diproses dalam string literal yang ditampilkan oleh *shell* interaktif Python atau sel keluaran Jupyter. Agar dapat diproses, kita perlu memberikan string tersebut ke fungsi `print`. Contoh berikut ini menunjukkan input-output dari karakter `escape` yang tidak diproses menjadi makna yang diharapkan:

```
'Li\tNa\t\tRb\n3\t11\t19\t37'
```

Output:

```
'Li\tNa\t\tRb\n3\t11\t19\t37'
```

Sementara itu, jika kita menggunakan fungsi `print`, kita akan memperoleh apa yang kita kehendaki dari karakter `escape`:

```
print('Li\tNa\t\tRb\n3\t11\t19\t37')
```

Output:

```
Li      Na      Rb
3       11      19      37
```

Pada contoh di atas, karakter *escape* `\t` telah dicetak sebagai tab dan `\n` mengakhiri baris keluaran dan memulai baris baru.

Ada kalanya (misalnya, saat mendefinisikan string kode sumber LaTeX), kita tidak ingin *backslash* dianggap sebagai karakter *escape*. Untuk kasus ini, kita perlu melakukan *escape* pada *backslash* itu sendiri (`'\\'`) atau definisikan *raw string* (`r'...'`) sebagai berikut:

```
print('\tan(\pi)')
# Ups: kita tidak ingin \t diubah menjadi tab
```

Output:

```
        an(\pi)
```

```
# Lakukan escape pada backslash
print('\\tan(\pi)')
```

Output:

```
\tan(\pi)
```

```
# Atau gunakan raw string
print(r'\tan(\pi)')
```

Output:

```
\tan(\pi)
```

Fungsi `print` sangatlah berguna dan dapat menerima urutan objek yang dipisahkan koma (termasuk angka, yang akan dikonversi menjadi string) lalu mencetaknya. Secara *default*, fungsi ini memisahkan objek-objek tersebut dengan satu spasi, dan memberikan satu baris baru (*newline*) di akhir keluarannya.

```
print('Standard atomic weight of Na:', 22.99)
```

Output:

```
Standard atomic weight of Na: 22.99
```

```
print('2 + 3 =', 2 + 3)
```

Output:

```
2 + 3 = 5
```

Perbedaan antara `\n` dan `\r` adalah bahwa `\r` mengembalikan "kursor" cetak ke awal baris tetapi tidak membuat baris baru, sehingga keluaran berikutnya menimpa apa pun yang ada di sana sebelumnya:

```
print("Fluorine\rChl")
print("Fluorine\nChl")
```

Output:

```
Chlorine
Fluorine
Chl
```

Karakter Unicode dapat diidentifikasi berdasarkan *code point*-nya: setiap karakter di hampir semua bahasa tulis utama di dunia diberikan bilangan bulat dalam standar Unicode, dan kode bilangan bulat ini dapat digunakan dalam string Python menggunakan kode *escape* `\u`:

```
print("\u212b") # Angstrom Swedia
```

Output:

```
Å
```

Pengindeksan dan Pemotongan String (*Slicing*)

Setiap karakter individual dalam suatu string dapat diakses menggunakan notasi pengindeksan `s[i]`, dengan indeks `i` adalah posisi karakter dalam string dimulai dari nol. Misalnya:

```
s = 'Plutonium'
s[0]
```

Output:

```
'P'
```

```
s[4]
```

Output:

```
'o'
```

Jika indeks `i` bernilai negatif, pengindeksan dihitung mundur dari akhir string (karakter terakhir berada di "posisi" -1):

```
s[-1]
```

Output:

```
'm'
```

```
s[-3]
```

Output:

```
'i'
```

Berikut adalah ilustrasi indeks pada string "Plutonium":

Karakter	P	l	u	t	o	n	i	u	m
Indeks	0	1	2	3	4	5	6	7	8
Indeks (-)	-9	-8	-7	-6	-5	-4	-3	-2	-1

Substring didapatkan dengan memotong (*slicing*) string menggunakan notasi `s[i:j]`, dengan `i` adalah indeks karakter pertama yang digunakan, dan `j` adalah satu posisi setelah karakter terakhir yang diambil. Artinya, substring mencakup karakter awal tapi mengecualikan karakter akhir:

```
s[2:4]
# Substring adalah s[2] + s[3], karakter ketiga dan keempat
```

Output:

```
'ut'
```

Untuk sepenuhnya memahami substring, kita perlu pembiasaan, tetapi sebenarnya notasinya cukup berguna. String yang dikembalikan memiliki $j - i$ karakter di dalamnya, dan potongan berurutan menggunakan indeks yang dimulai tepat di tempat potongan sebelumnya berakhir:

```
s[0:4] + s[4:7] + s[7:9]
```

Output:

```
'Plutonium'
```

String dapat dipotong bahkan dihitung mundur atau dicacah dengan langkah (*stride*) yang berbeda menggunakan nilai ketiga dalam format `s[i:j:stride]`. Jika salah satu dari `i`, `j`, atau `stride` dihilangkan (dengan tetap menjaga format tanda titik dua), interpreter Python akan mengasumsikan nilai berturut-turut adalah awal string ($i = 0$), akhir string, dan stride $= 1$ (karakter berurutan). Perhatikan beberapa contoh berikut.

```
s[:5]
# Lima karakter pertama
```

Output:

```
'Pluto'
```

```
s[1:6:2]
# s[1], s[3], s[5]
```

Output:

```
'ltn'
```

```
s[::-1]
# Seluruh string, dibalik
```

Output:

```
'muinotulp'
```

String dalam Python bersifat *immutable* (tidak dapat diubah). Kita tidak dapat mengubahnya seketika dalam proses interpretasi sehingga upaya perubahan akan memberikan pesan *error*.

```
s = 'alkane'
s[3] = 'e'
```

Output:

```
TypeError                                 Traceback (most recent call last)
Cell In[4], line 2
      1 s = 'alkane'
----> 2 s[3] = 'e'

TypeError: 'str' object does not support item assignment
```

Sebagai gantinya, string baru dapat dibuat dari potongan string yang lama:

```
# 'alk' + 'e' + 'ne'
s[:3] + 'e' + s[4:]
```

Output:

```
'alkene'
```

String baru tersebut tentu saja dapat ditetapkan ke nama variabel, termasuk variabel yang sama, jika kita kehendaki.

```
# 'alk' + 'y' + 'ne'
s = s[:3] + 'y' + s[4:]
s
```

Output:

```
'alkyne'
```

2 Metode String

Python dilengkapi dengan sejumlah besar metode untuk memanipulasi string. Daftar lengkapnya tersedia dalam dokumentasi resmi Python. Di sini kita akan bahas beberapa metode yang paling berguna.

```
# definisikan string yang akan dijadikan contoh:
molekul = 'CH3CH2CH2CH3'

# panjang urutan, fungsi bawaan (built-in) Python:
len(molekul)
```

Output:

```
12
```

```
# jumlah substring
molekul.count('CH2')
```

Output:

```
2
```

```
# mengubah semua karakter menjadi huruf kecil
molekul.lower() # selain ini, tersedia juga molekul.upper()
```

Output:

```
'ch3ch2ch2ch3'
```

```
# menentukan indeks pertama dari substring
molekul.index('CH2')
```

Output:

```
3
```

```
# hapus karakter 'C', 'H', '3' mana pun dari kedua ujung f
molekul.strip('CH3')
```

Output:

```
'2CH2'
```

```
molekul.removeprefix('CH3') # selain itu, ada juga molekul.removesuffix()
↪  sejak Python 3.9
```

Output:

```
'CH2CH2CH3'
```

```
molekul.replace('CH3', 'NH2')
```

Output:

```
'NH2CH2CH2NH2'
```

Sebagai penegasan, string `f` bersifat *immutable* dan tidak berubah oleh operasi-operasi ini. Apa yang dilakukan operasi-operasi tersebut adalah mengembalikan objek Python baru. Dengan kata lain, metode-metode tersebut dapat dirangkai (*chained*) bersama, misalnya:

```
molekul.removeprefix('CH3').removesuffix('CH3').lower()
```

Output:

```
'ch2ch2'
```

3 Pemformatan String

Ketika sebuah angka dicetak ke layar, Python mengubahnya menjadi string dan berusaha sebaik mungkin untuk memilih representasi yang sesuai. Bilangan bulat selalu ditampilkan dengan semua digitnya, sementara untuk bilangan *floating point* yang sangat besar atau sangat kecil, Python mengadopsi notasi ilmiah.

```
N_A = 602_214_076_000_000_000_000_000
print(N_A)
```

Output:

```
602214076000000000000000
```

```
print(float(N_A))
```

Output:

```
6.02214076e+23
```

Cara angka diformat dapat dikontrol menggunakan metode string `format`. Dalam penggunaan paling sederhana, nilai-nilai disisipkan begitu saja ke dalam *template* string pada lokasi yang ditunjukkan oleh kurung kurawal (`'{}'`):

```
c = 299792458
units = 'm.s-1'
'The speed of light is c={} {}'.format(c, units)
```

Output:

```
'The speed of light is c=299792458 m.s-1'
```

Argumen yang diteruskan ke metode `format` juga dapat dirujuk berdasarkan nama atau indeks (berbasis nol) di dalam kurung kurawal:

```
'{0}={1} {cgs_units}'.format('c', c*100, cgs_units='cm.s-1')
```

Output:

```
'c=29979245800 cm.s-1'
```

Ada sintaksis khusus untuk lebih memperindah pemformatan string hasil interpolasi ini, di antaranya adalah lebar bidang (*field width*), *padding*, jumlah tempat desimal, dan sebagainya. Sebagai contoh, untuk bilangan bulat, lebar spasi yang dialokasikan untuk angka (yakni `w`) ditentukan dengan `:wd`.

```
'{:6d}'.format(42)
```

Output:

```
'    42'
```

Angka 42 dikeluarkan sebagai string sejumlah 6 karakter, diawali dari sebelah kiri dengan beberapa spasi hingga akhirnya angka 2 merupakan karakter ke-6 (indeks 5) dalam string tersebut.

Untuk bilangan riil *floating-point*, kita dapat menentukan presisi, `p`, serta lebar, `w`, yakni `:w.pf` dan `:w.pe`, masing-masing untuk notasi posisi dan notasi ilmiah.

```
'{:.28f}'.format(k_B) # notasi posisi, 28 karakter
```

Output:

```
'0.000000000000000000000138065'
```

```
k_B = 1.380649e-23
'{:12.3e}'.format(k_B) # notasi ilmiah 12 karakter, 3 tempat desimal
```

Output:

```
'   1.381e-23'
```

```
'{:.3e}'.format(k_B) # karakter sebanyak yang dibutuhkan, 3 desimal
```

Output:

```
'1.381e-23'
```

Perhatikan bahwa jika lebar dihilangkan, Python membuat string sepanjang yang diperlukan untuk memberikan presisi yang diminta.

Ada beberapa jenis penentu format (*format specifier*) lainnya yang berguna. Di antaranya, `:w.pg` dapat beralih antara notasi posisi dan ilmiah tergantung pada besarnya angka dan presisi yang diminta. Kita bisa juga menggunakan `:ws` untuk menginterpolasi string ke dalam string lain (yang terkadang diperlukan untuk menyelaraskan output).

Sejak Python versi 3.6, kita dapat menginterpolasi nilai objek secara langsung menggunakan *f-strings*. Nama variabel objek ditempatkan sebelum titik dua di dalam string yang didefinisikan dalam tanda kutip yang didahului oleh huruf `f` sebagai berikut (tanda petik ganda akan digunakan sebagai awalan dan penutup karena ada satu petik tunggal sebagai apostrof):

```
f"Boltzmann's constant is approximately {k_B:.3e} J.K-1"
```

Output:

```
"Boltzmann's constant is approximately 1.381e-23 J.K-1"
```

Kita bahkan bisa juga melakukan perhitungan di dalam kurung kurawal interpolasi. Artinya, objek yang diinterpolasi dapat berasal dari sebuah ekspresi. Untuk menjaga kode tetap jelas, lebih baik tidak menyertakan ekspresi kompleks dalam *f-string*, tetapi boleh saja dilakukan misalnya ketika melakukan konversi satuan.

```
print(f'The speed of light is approximately {c/1000:.3f} km.s-1')
```

Output:

```
The speed of light is approximately 299792.458 km.s-1
```

Studi Kasus

Kasus 1 Massa Alkana

Buatlah string terformat yang memberikan massa rantai alkana linear C_nH_{2n+2} jika diberikan n. Gunakan massa molar C dan H berturut-turut sebesar 12.0107 g.mol^{-1} dan 1.00784 g.mol^{-1}.

Solusi

Kita definisikan terlebih dahulu massa molarnya dan *template* string yang akan digunakan:

```
# Massa molar atom karbon dan hidrogen (g.mol-1).
mC, mH = 12.0107, 1.00784

# Template pemformatan string.
fmt = 'Mass of {:5s} : {:6.2f} g.mol-1'
```

Selanjutnya kita hitung untuk $n = 2$ dan $n = 8$:

```
n = 2
nH = 2*n + 2
formula = 'C{}H{}'.format(n, nH) # atau f'C{n}H{nH}'
print(fmt.format(formula, n*mC + nH*mH))

n = 8
nH = 2*n + 2
formula = 'C{}H{}'.format(n, nH)
print(fmt.format(formula, n*mC + nH*mH))
```

Output:

```
Mass of C2H6  :  30.07 g.mol-1
Mass of C8H18 : 114.23 g.mol-1
```

Kasus 2 Beberapa Konstanta Fisis

Buatlah daftar nilai konstanta fisis h, c, k_B, R, dan N_A yang diformat dengan rapi hingga empat angka penting, beserta satuannya.

Solusi

Pertama, definisikan variabel untuk nilai dan satuan konstanta fisis tersebut.

```
h, h_units = 6.62607015e-34, 'J.s'
c, c_units = 299792458, 'm.s-1'
kB, kB_units = 1.380649e-23, 'J.K-1'
R, R_units = 8.314462618, 'J.K.mol-1'
N_A, N_A_units = 6.02214076e+23, 'mol-1'
```

Kita dapat mendefinisikan daftar konstanta dalam satu string melalui penggabungan (*concatenation*). Kita bisa juga memecah penetapan (*assignment*) konstanta ke dalam beberapa baris input, tetapi sisi kanan baris terakhir harus dibungkus dalam tanda kurung agar Python menafsirkannya sebagai ekspresi tunggal.

```
s = (f'h   = {h:9.3e} {h_units}\n'
     f'c   = {c:9.3e} {c_units}\n'
     f'kB  = {kB:9.3e} {kB_units}\n'
     f'R   = {R:9.3f} {R_units}\n'
     f'N_A = {N_A:9.3e} {N_A_units}')
print(s)
```

Output:

```
h   = 6.626e-34 J.s
c   = 3.000e+08 m.s-1
kB  = 1.381e-23 J.K-1
R   =     8.314 J.K.mol-1
N_A = 6.022e+23 mol-1
```

Kasus 3 Pemformatan Data Termodinamika

Variabel-variabel berikut mendefinisikan beberapa sifat termodinamika dari CO_2 dan H_2O

```
# Titik tripel CO2 (K, Pa).
T3_CO2, p3_CO2 = 216.58, 5.185e5
# Entalpi peleburan CO2 (kJ.mol-1).
DfusH_CO2 = 9.019
# Entropi peleburan CO2 (J.K-1.mol-1).
DfusS_CO2 = 40
# Entalpi penguapan CO2 (kJ.mol-1).
DvapH_CO2 = 15.326
# Entropi penguapan CO2 (J.K-1.mol-1).
DvapS_CO2 = 70.8

# Titik tripel H2O (K, Pa).
T3_H2O, p3_H2O = 273.16, 611.73
# Entalpi peleburan H2O (kJ.mol-1).
DfusH_H2O = 6.01
# Entropi peleburan H2O (J.K-1.mol-1).
DfusS_H2O = 22.0
# Entalpi penguapan H2O (kJ.mol-1).
DvapH_H2O = 40.68
# Entropi penguapan H2O (J.K-1.mol-1).
DvapS_H2O = 118.89
```

Gunakan serangkaian perintah `print` dengan *f-strings* untuk menghasilkan tabel terformat semacam ini:

		CO2	H2O
p3	/Pa	518500	611.73
T3	/K	216.58	273.16
DfusH	/kJ.mol-1	9.019	6.010
DfusS	/J.K-1.mol-1	40.0	22.0
DvapH	/kJ.mol-1	15.326	40.680
DvapS	/J.K-1.mol-1	70.8	118.9

Solusi

Kita perlu mengatur lebar kolom dan presisi desimal agar sesuai dengan tabel target. Sebagai contoh, perhatikan bahwa nilai input `118.89` untuk `DvapS_H2O` dibulatkan menjadi `118.9` dalam tampilan tabel, yang menunjukkan penggunaan format `.1f`. Format string lainnya dapat kita sesuaikan.

```python
# Header Tabel
print(f"{'':<6} {'':<14} {'CO2':>10} {'H2O':>10}")

# Baris p3 (Pa): CO2 integer, H2O 2 desimal
print(f"{'p3':<6} {'/Pa':<14} {p3_CO2:10.0f} {p3_H2O:10.2f}")

# Baris T3 (K): 2 desimal
print(f"{'T3':<6} {'/K':<14} {T3_CO2:10.2f} {T3_H2O:10.2f}")

# Baris Entalpi Peleburan (kJ.mol-1): 3 desimal
print(f"{'DfusH':<6} {'/kJ.mol-1':<14} {DfusH_CO2:10.3f}
↪  {DfusH_H2O:10.3f}")

# Baris Entropi Peleburan (J.K-1.mol-1): 1 desimal
print(f"{'DfusS':<6} {'/J.K-1.mol-1':<14} {DfusS_CO2:10.1f}
↪  {DfusS_H2O:10.1f}")

# Baris Entalpi Penguapan (kJ.mol-1): 3 desimal
print(f"{'DvapH':<6} {'/kJ.mol-1':<14} {DvapH_CO2:10.3f}
↪  {DvapH_H2O:10.3f}")

# Baris Entropi Penguapan (J.K-1.mol-1): 1 desimal
print(f"{'DvapS':<6} {'/J.K-1.mol-1':<14} {DvapS_CO2:10.1f}
↪  {DvapS_H2O:10.1f}")
```

Output:

```
                                CO2        H2O
p3     /Pa                   518500     611.73
T3     /K                    216.58     273.16
DfusH  /kJ.mol-1              9.019      6.010
DfusS  /J.K-1.mol-1            40.0       22.0
DvapH  /kJ.mol-1             15.326     40.680
DvapS  /J.K-1.mol-1            70.8      118.9
```